
VULNERABILITY ASSESSMENT REPORT

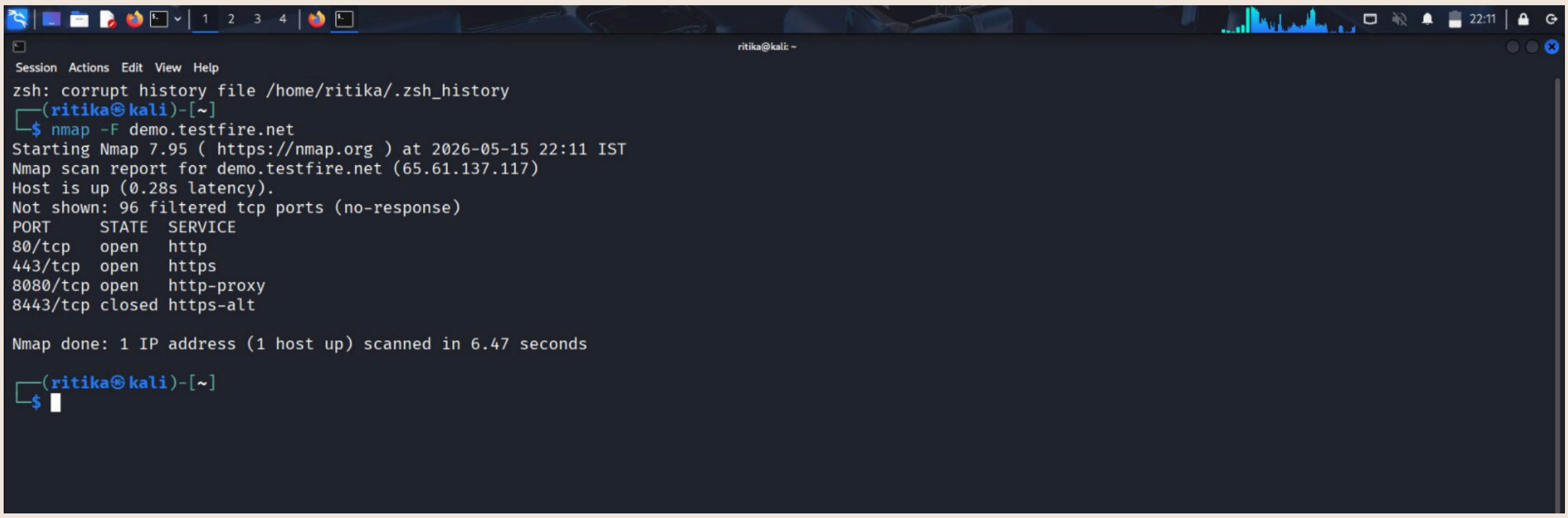
- **Target Website**: demo.testfire.net
- **Prepared by**: Ritika Karak
- **Tools Used:** Nmap, OWASP ZAP, Browser Dev Tools
- **Date:** 16/05/2026

Executive Summary

The primary goal of this report was to conduct a security "**health check**" on the target website. We aimed to identify any "**holes**" or weaknesses that a hacker could use to steal user data or disrupt the service. By using industry-standard tools, we analyzed the site from different angles —from its server ports to its internal browser settings.

Phase 1

Port Scanning (Nmap)



- **Description**: Think of ports as digital doors to a house. We used Nmap to see which doors are open to the public. We performed a "Service Version Detection" and "Fast Scan" using Nmap to identify which active services are exposed to the internet.

- **Observation**: *Open Ports*: We found that Port 80 (HTTP) and Port 443 (HTTPS) are open. This is normal for any website.

- **Analysis**: While these are necessary for web traffic, having 98 other ports "filtered" indicates that a firewall is active, which is a good defensive measure.

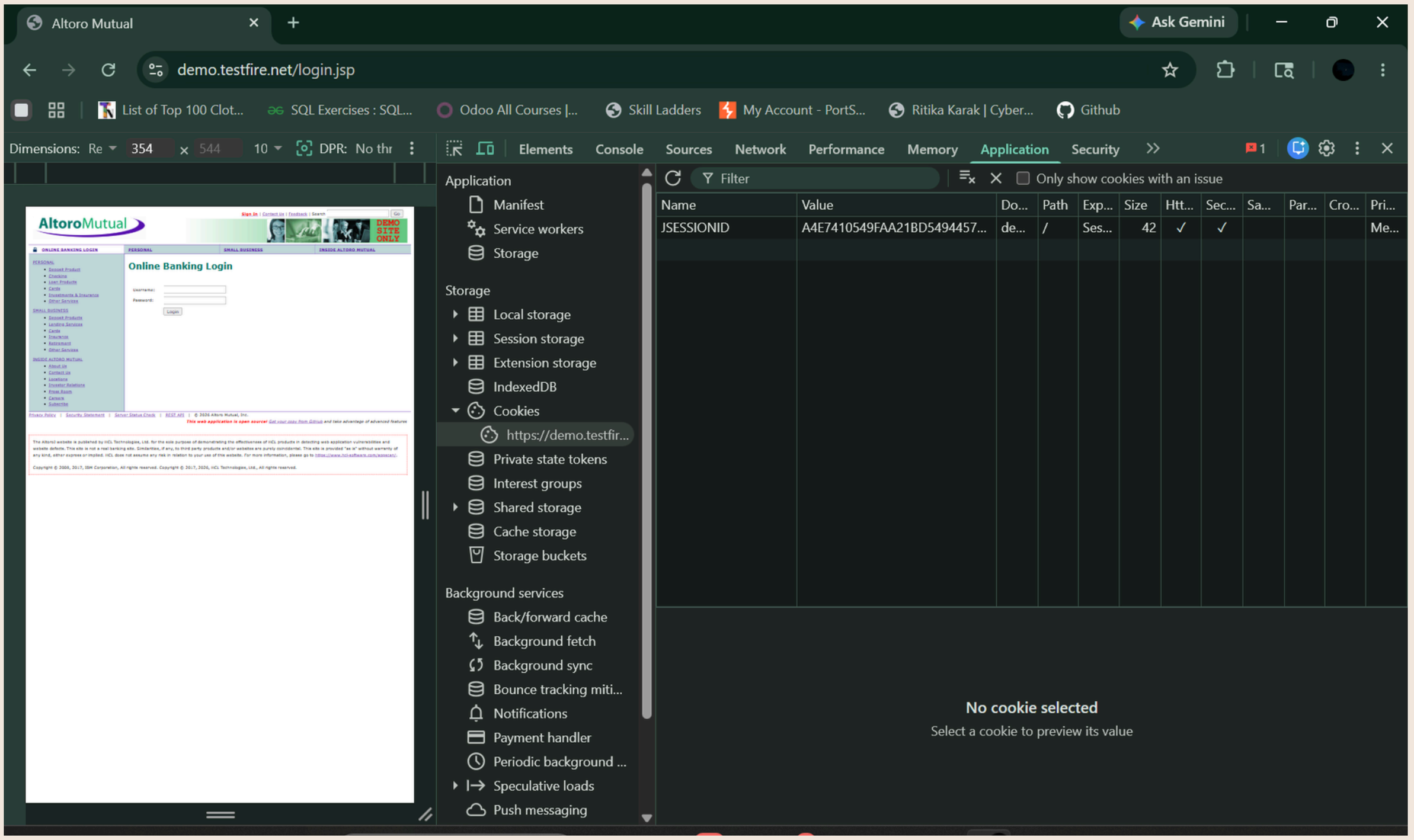
- **The Risk**: If ports like 21 (FTP) or 22 (SSH) were open without strict access control, an attacker could attempt "Brute Force" attacks to gain server access.If a firewall is not strictly configured, "Port Scanning" allows hackers to perform "OS Fingerprinting," helping them tailor their attacks to the specific operating system the server is running.

Solution (Remediation):

1. **Port Hardening**: Close all ports that are not strictly required for business operations (e.g., FTP, SSH, or Database ports should not be public).
2. **Firewall Configuration:** Use a Web Application Firewall (WAF) to filter out malicious traffic before it reaches these open ports.
3. **Disable Banner Grabbing**: Configure the server to hide its version identity. For Apache, set ServerTokens Prod and ServerSignature Off in the configuration file.

Phase 2

Cookie Security Analysis (Browser DevTools)



- **Description**: Cookies are small bits of data saved in your browser (like your login session). If these aren't locked properly, a hacker can steal them to log into your account.

- **Observation**:We found that the JSESSIONID cookie has both **HttpOnly** and **Secure** flags enabled.

1. **Why HttpOnly?** Without this, a hacker could use a Cross-Site Scripting (XSS) attack to run a script like document.cookie and steal the user's session ID instantly.

2. **Why Secure?** It ensures the cookie is only transmitted over an encrypted HTTPS connection.This ensures the cookie is never sent over an unencrypted connection, protecting the user from "Man-in-the-Middle" (MITM) attacks on public Wi-Fi.

- **The Risk**:If these flags were missing, a hacker could perform **Session Hijacking**, allowing them to log into the user's account without ever knowing their password.

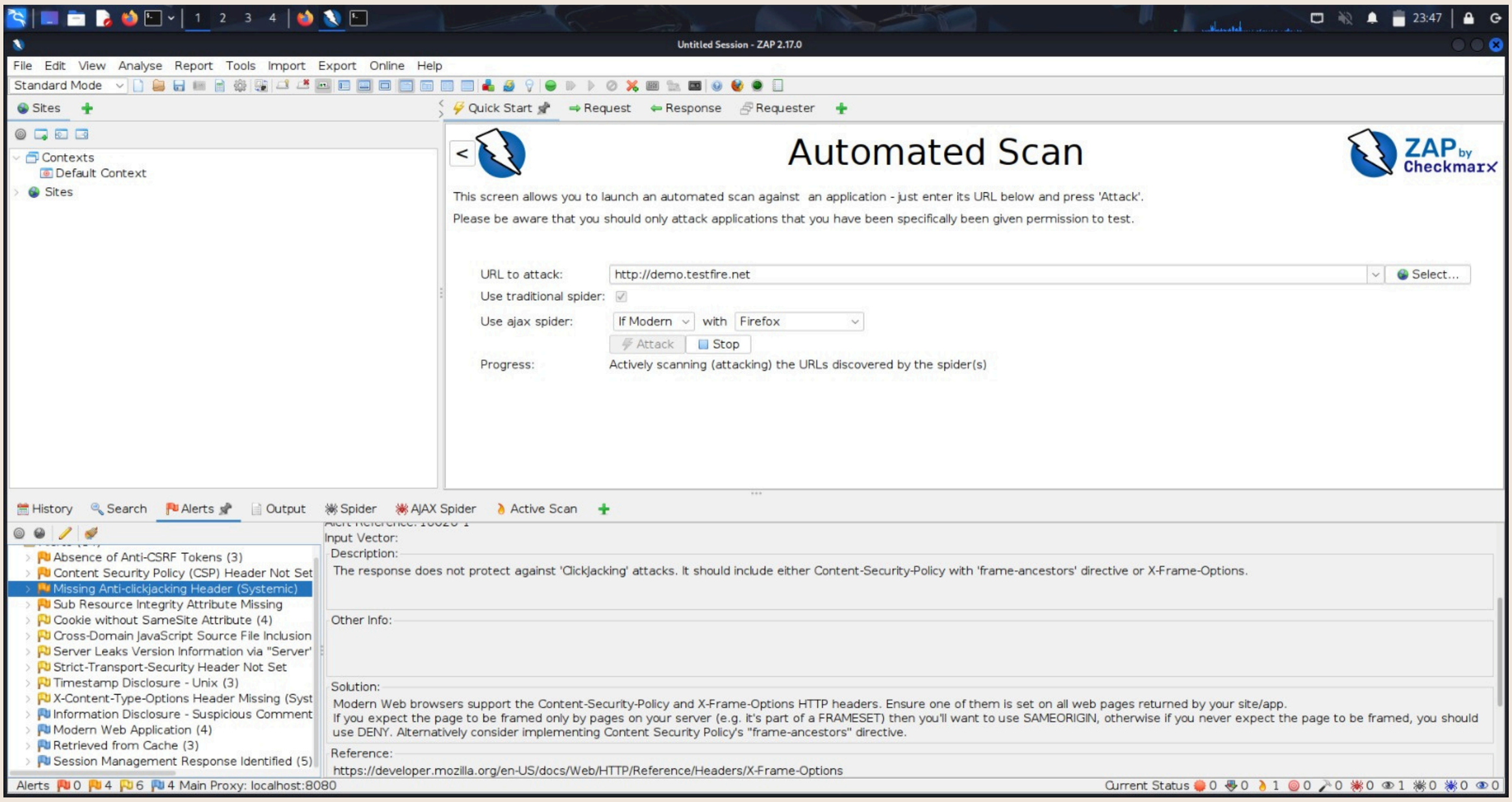
.....

Solution (Remediation):

SameSite Attribute: Developers should also add the SameSite=Strict or Lax attribute. This prevents **Cross-Site Request Forgery (CSRF)** by ensuring the cookie is only sent for requests originating from the same website.

Phase 3

Vulnerability Scanning (OWASP ZAP)



- **Description**: We used OWASP ZAP to automatically scan the site for deeper security flaws.

A. Issue Name: Anti-Clickjacking Header Not Set

- ***Risk Level***: Medium 🟠
- ***Description***: The server does not send the X-Frame-Options header. This header controls whether a browser is allowed to render a page in an <frame> or <iframe>.
- ***Impact***: Attackers can perform a Clickjacking attack by embedding this website into a malicious site. Users might be tricked into performing unintended actions (like deleting data) while thinking they are interacting with a different site.

- ***Remediation***: 1. Open your web server configuration (e.g., .htaccess or nginx.conf). 2. Add the following line: X-Frame-Options: SAMEORIGIN. 3. Restart the server to apply the change.

B. Issue Name: X-Content-Type-Options Header Missing

- ***Risk Level:*** Low 🟡
- ***Description:*** The X-Content-Type-Options: nosniff header is missing. This prevents the browser from "MIME-sniffing" the content type of a file.

- ***Impact***: An attacker could upload a malicious script disguised as a different file type (e.g., an image). If the browser guesses the type and runs it as a script, it could lead to an **XSS (Cross-Site Scripting)** attack.
- ***Remediation***: 1. Configure the server to include the header: X-Content-Type-Options: nosniff in all HTTP responses.

C. Issue Name: Insecure Communication (Information Leakage)

- ***Risk Level*:** Low 🟡

- ***Description:*** The application allows communication over plain HTTP (Port 80) without automatically redirecting to HTTPS.
- ***Impact*:** Data transmitted over HTTP is not encrypted. An attacker on the same network (e.g., public Wi-Fi) could perform a **Man-in-the-Middle (MITM)** attack to sniff user credentials.
- ***Remediation*:** 1. Implement a global redirect from HTTP to HTTPS. 2. Enable **HSTS (HTTP Strict Transport Security)** to force browsers to always use secure connections.

Final Recommendations

- **Hardening Security Headers**: Implement a strong **Content Security Policy (CSP)** to prevent unauthorized script execution.
- **Server Updates**: Regularly update the web server software (Apache/Tomcat) to the latest version to patch known vulnerabilities.
- **Input Sanitization**: Ensure all user inputs are validated on the server side to prevent injection attacks.